

PROJECT DOCUMENTATION

HOSPITAL OPERATION AND PATIENT ANALYSIS

Hospital Operations & Patient Analytics Dashboard Suite

Offered by

INFOSYS SPRINGBOARD VIRTUAL INTERNSHIP 7.0
BATCH 2

Submitted by: Meera Jasmine Shaik

Project Details

Project Title	Hospital Operation and Patient Analysis
Program	Infosys Springboard Virtual Internship 7.0 - Batch 2
Candidate Name	Meera Jasmine Shaik
Mentor Name	Pathina Likhita
Internship Start Date	28 July 2026
Internship End Date	17 Sep 2026
Platforms / Tools Used	Jupyter Notebook, Tableau, Power BI

Acknowledgement

I would like to express my sincere gratitude to **Infosys Springboard** for providing me the opportunity to undertake this virtual internship (Batch 2) and work on the project **“Hospital Operation and Patient Analysis.”** This experience has given me valuable exposure to real-world data analytics workflows, from data collection and cleaning to building interactive business intelligence dashboards.

I am extremely thankful to my mentor **Pathina Likhita** for their continuous guidance, valuable feedback, and encouragement throughout the course of this internship. Their support was instrumental in helping me understand healthcare operations analytics and in successfully completing each milestone of this project.

I would also like to thank the entire Infosys Springboard team for structuring this internship program in a way that allowed hands-on learning with tools such as Python, Tableau, and Power BI. This project has significantly strengthened my skills in data cleaning, KPI engineering, and dashboard development.

Finally, I extend my heartfelt thanks to my family, friends, and peers for their constant support and motivation during this internship journey.

- **Meera Jasmine Shaik**

Index

Module

Module 1: Healthcare Data Collection 5–6

- Sourcing and organizing the five raw hospital operation files
- Reviewing structure, row counts and column meaning of each file
- Defining the role each dataset plays in the final analysis

Module 2: Data Cleaning & Transformation..... 7–8

- Null-value, duplicate and ID-uniqueness audits
- Column-name standardization and date parsing
- Service-level, staff and attendance aggregation
- Merging everything into the final combined dataset

Module 3: Hospital KPI Engineering 9–10

- Length of Stay, Total Admissions and Average Length of Stay
- Bed Utilization Rate and Occupancy Rate
- Readmission Rate and the service-level KPI table

Module 4: Dashboard Planning & Prototyping..... 11–12

- Wireframes for all four dashboard pages
- KPI, filter and visual-type planning per page
- Navigation and interactivity design

Module 5: Build Patient Analytics Dashboard (Tableau & Power BI)..... 13–15

- KPI scorecards and condition / cost visuals
- Patient geography and admission-trend visuals
- Chart-by-chart explanation of every visual used

Module 6: Build Staff Management & Service Analytics Dashboard (Tableau & Power BI) 16–18

- Staffing distribution and outlier detection visuals
- Service-demand and satisfaction-by-service visuals
- Chart-by-chart explanation of every visual used

Module 7: Build Financial Overview & Patient Insights Dashboard (Tableau & Power BI) 19–21

- Cost, insurance-claim and geographic-spend visuals
- Bed-utilization and patient-outcome flow visuals
- Chart-by-chart explanation of every visual used

Module 1: Healthcare Data Collection

Milestone 1 — Data Collection and Preparation

The first module focused on identifying and organizing the raw hospital operations and patient data needed to build the analytics suite. Rather than working from a single dataset, the project deliberately combined five separate files that together capture the patient side (who was admitted, for what condition, at what cost) and the operations side (bed capacity, staffing, and attendance) of a hospital network. Keeping these as separate source files at the collection stage, and only merging them later in Module 2, mirrors how real hospital data is typically stored across different departmental systems.

Datasets Collected

Five raw CSV files were placed in the Module 1 folder. Each one is described below along with the fields it contributes and why it matters to the overall analysis.

- **Hospital Patient.csv** — 984 patient records across 15 columns: Patient_ID, Age, Gender, Condition, Medication, Admission_Date, Discharge_Date, Patient_State, Year_of_Admission, Length_of_Stay, Readmission, Outcome, Satisfaction, Insurance_Claimed and Total_Cost. This is the richest patient-level file and feeds most of the clinical and financial visuals — condition mix, cost by condition, readmission, outcome, and geography.
- **patients.csv** — 1,000 patient visit records with patient_id, name, age, arrival_date, departure_date, service and satisfaction. This file links individual patients to a specific hospital service (Emergency, General Medicine, ICU, Surgery) and provides the arrival/departure timestamps used to calculate Length of Stay in Module 3.
- **services_weekly.csv** — 208 weekly records per service, with available_beds, patients_request, patients_admitted, patients_refused, patient_satisfaction, staff_morale and event. This is the operational backbone of the project — it is what makes bed-utilization, occupancy, and demand-vs-capacity analysis possible.
- **staff.csv** — 110 staff records with staff_id, staff_name, role (doctor / nurse) and service. Used to calculate staffing levels and doctor-to-nurse ratios per department.
- **staff_schedule.csv** — 6,552 attendance records with week, staff_id, staff_name, role, service and a present flag. This is the largest file in the project and is aggregated into weekly/overall attendance rates per service.

Data Collection Methodology

The collection step was more than simply downloading files — each dataset was opened and manually inspected before being accepted into the pipeline, to make sure it could realistically be joined with the others later. The key checks performed at this stage were:

- Confirming that every file had a common linking key — either patient_id/Patient_ID or service — so the datasets could eventually be merged without losing records.
- Checking that categorical values (service names such as “emergency”, “general_medicine”, “ICU”, “surgery”) were spelled and cased consistently enough across files to be joined cleanly.
- Verifying that every file loaded into Pandas without parsing errors and that row/column counts matched expectations.
- Noting which fields would need type conversion later (dates stored as text, percentages stored as decimals, etc.) so that Module 2 could address them systematically.

Why This Data Matters

Hospital administrators rarely need just one view of the truth. Patient-level data (Hospital Patient.csv, patients.csv) answers “who are we treating and what does it cost?”, while operations-level data (services_weekly.csv, staff.csv, staff_schedule.csv) answers “can we handle the load, and is our staff coping?”. Collecting both sides separately at this stage, before any cleaning or merging, ensured that neither the clinical perspective nor the operational perspective was diluted when they were eventually combined into the single analytics-ready dataset used in the dashboards.

Deliverables

- Hospital Patient.csv, patients.csv, services_weekly.csv, staff.csv, staff_schedule.csv (Module 1 folder)

Evaluation / Outcome

- All five datasets were successfully collected, opened, and verified as structurally compatible.
- Dataset completeness and join-key consistency confirmed prior to cleaning in Module 2.

Tools & Techniques Used

Data collection was carried out entirely in a Python/Jupyter environment, with the raw CSV files first opened in Excel for a manual spot-check before being loaded into Pandas for a more rigorous inspection. This two-step approach — a quick human read-through followed by a programmatic check — caught several minor issues, such as stray whitespace in column headers and inconsistent capitalization in service names, before they could propagate into later modules.

- **Pandas (pd.read_csv, .shape, .columns, .dtypes)** — used to load every file and produce an at-a-glance structural summary.
- **Excel** — used for an initial manual review of each raw file, particularly to visually confirm column headers and spot obvious data-entry errors.
- **A shared folder structure (Module 1 through Module 7)** — used to keep raw, cleaned, and KPI-enriched versions of the data clearly separated throughout the project.
- **A short data dictionary** — drafted for each file, recording column name, data type and a one-line description, to speed up the cleaning work in Module 2.

Challenges & Resolutions

Two practical challenges surfaced during collection. First, the same real-world entity was represented differently across files — patients were identified as Patient_ID in one file and patient_id in another, and departments were written as “ICU” in one file but “icu” in another. Second, the five files varied enormously in size, from 110 rows (staff.csv) to 6,552 rows (staff_schedule.csv), which meant the eventual merge in Module 2 needed careful thought about which table should sit on the “one” side and which on the “many” side of each join.

- **Naming inconsistency across files** — resolved in Module 2 with a dedicated clean_column_names() function rather than fixing each file individually.
- **Case-sensitivity in service names** — flagged at the collection stage and confirmed safe to standardize with a simple lower-casing pass.
- **Large size disparity between files** — addressed by aggregating the weekly and attendance files to one row per service before merging, rather than merging at the raw row level.

Module 2: Data Cleaning & Transformation

Milestone 1 — Data Collection and Preparation

This module cleaned, standardized, and combined the five raw datasets into a single analysis-ready table. All of the work was carried out in Python inside code.ipynb using Pandas and NumPy. The notebook follows a clear, auditable sequence — inspect, validate, clean, aggregate, merge — so that every transformation applied to the data can be traced back to a specific step.

Step-by-Step Methodology

Step 1: Load and inspect. All five CSV files were loaded into Pandas DataFrames and stored in a dictionary keyed by dataset name. For each dataset, the row count, column count, and full column list were printed, giving an at-a-glance inventory before any cleaning began.

Step 2: Null-value audit. For every dataset, a null-value report was generated showing the null count and null percentage for each column. This made it possible to see exactly which fields (if any) needed imputation or special handling before they were used downstream.

Step 3: Duplicate-row check. Each dataset was checked for fully duplicated rows using `df.duplicated().sum()`, confirming that no dataset contained accidental repeat records that would inflate counts later.

Step 4: ID uniqueness check. `Patient_ID` (Hospital Patient), `patient_id` (patients) and `staff_id` (staff) were each checked by comparing the total ID count against the count of unique IDs, to confirm that identifiers were behaving as true primary keys.

Step 5: Category-value consistency check. The unique service values in `patients.csv`, `services_weekly.csv` and `staff.csv` were printed side by side and compared, to make sure department names (emergency, general_medicine, ICU, surgery) matched exactly across files before being used as a merge key.

Step 6: Column-name standardization. A reusable `clean_column_names()` function was written that strips whitespace, lower-cases every column name, and replaces spaces and hyphens with underscores. It was applied to all five datasets so that column references would be consistent (e.g. `Patient_ID` and `patient_id` both became `patient_id`-style names) across the whole pipeline.

Step 7: Date parsing and validation. `admission_date` and `discharge_date` (Hospital Patient) and `arrival_date` / `departure_date` (patients) were converted to proper datetime objects using `pd.to_datetime()`, with `dayfirst=True` for the day-month-year formatted fields and `errors="coerce"` so that any unparseable value became a clear, checkable null rather than silently breaking the pipeline. A follow-up null check on the converted date columns confirmed how many, if any, dates failed to parse.

Step 8: Service-level aggregation. `services_weekly.csv` was grouped by service and aggregated into `total_available_beds`, `total_patient_requests`, `total_patients_admitted`, `total_patients_refused`, `avg_patient_satisfaction` and `avg_staff_morale` — turning 208 weekly rows into one summary row per department.

Step 9: Bed-utilization calculation. A `bed_utilization_rate` column was derived as `total_patients_admitted ÷ total_available_beds`, then clipped at an upper bound of 1 (100%) to guard against any data-entry edge cases where admissions could exceed reported capacity in a given aggregation.

Step 10: Staff summary. `staff.csv` was grouped by service to compute `total_staff` (unique `staff_id` count), and separate doctors and nurses counts using conditional aggregation on the `role` column.

Step 11: Attendance summary. `staff_schedule.csv` (6,552 rows) was grouped by service into `total_staff_schedule_records`, `total_present` and an `attendance_rate` (mean of the present flag, converted to a percentage).

Step 12: Combine the operational tables. The service-level summary, staff summary, and attendance summary were merged together on the service key into a single `service_operations` table, giving one operationally complete row per department.

Step 13: Build the final dataset. The patients table was left-merged with `service_operations` on service, producing the final combined dataset that carries every patient's individual visit details alongside the operational context (capacity, staffing, attendance) of the department that treated them. The resulting shape was printed and reviewed before export.

Deliverables

- `code.ipynb` — data cleaning and transformation notebook
- `Hospital_Operations_Final_Dataset.xls` — cleaned, merged dataset ready for KPI engineering and Tableau/Power BI import

Evaluation / Outcome

- Null values, duplicates and ID inconsistencies identified and resolved prior to merging.
- Missing values reduced to well below the 2% completeness threshold.
- Consistent, standardized column names and operational metrics across all five source datasets.

Tools & Libraries Used

All thirteen cleaning and transformation steps were implemented in a single Jupyter notebook, `code.ipynb`, so the entire pipeline could be re-run end-to-end from the raw files to the final merged dataset with one click. Pandas handled almost every operation; NumPy was used sparingly, mainly for numeric clipping and conditional logic.

- **`pandas.merge()`** — used repeatedly to combine the service, staff and attendance summaries, and finally the patient-level data, always with an explicit `how=` argument so join behaviour was never left to a default.
- **`pandas.to_datetime()` with `errors="coerce"`** — used for every date column, so bad values became inspectable NaT entries instead of raising exceptions.
- **`groupby().agg()`** — used to collapse the 208-row `services_weekly.csv` and 6,552-row `staff_schedule.csv` into one summary row per service.
- **`numpy.clip()`** — used once, to cap the `bed_utilization_rate` at 100%.

Challenges & Resolutions

The main challenge in this module was deciding the correct order of operations — aggregating too early risked losing detail needed for later KPIs, while aggregating too late risked carrying inconsistent keys into the merge. The step-by-step methodology described above (inspect, validate, clean, aggregate, merge) was adopted specifically to keep this order explicit and auditable.

- **Date fields stored as day-first text strings** — resolved with the `dayfirst=True` argument rather than a manual string-parsing routine.
- **A small number of unparseable dates** — surfaced as NaT via `errors="coerce"` and reviewed individually rather than silently dropped.
- **Choosing a merge key for the final dataset** — resolved by merging on the standardized service column only, after confirming it held identical values in every table that needed to be joined.

Module 3: Hospital KPI Engineering

Milestone 2 — KPI Engineering and Dashboard Planning

Using the merged dataset produced in Module 2, this module engineered the core healthcare KPIs that power every dashboard in the suite. The work was carried out in `KPI_Engineering.ipynb`, and each KPI was derived with a simple, explainable formula so that the numbers shown on the dashboards can always be traced back to source columns.

KPI Definitions and Formulas

- **Length of Stay** — calculated per patient as `departure_date` minus `arrival_date`, expressed in days. Summary statistics (mean, min, max, quartiles) were reviewed with `.describe()` to sanity-check the distribution before using it elsewhere.
- **Total Admissions** — the count of unique `patient_id` values in the final dataset, giving the true number of distinct patients treated rather than the number of visit rows.
- **Average Length of Stay** — the mean of the `length_of_stay` column across all patients, reported in days and used as a headline KPI card on every dashboard page.
- **Bed Utilization Rate (overall)** — total patients admitted (summed across all services) divided by total available beds (summed across all services), multiplied by 100 to express it as a percentage.
- **Service-level KPI table** — a dedicated table (`service`, `total_available_beds`, `total_patients_admitted`) was built by de-duplicating the service-level rows, and a `bed_utilization_rate` was calculated for each individual service, letting the dashboards compare departments side by side rather than only showing one hospital-wide number.
- **Occupancy Rate** — computed the same way as bed utilization but aggregated across the service-level KPI table — the sum of patients admitted across all services divided by the sum of available beds across all services, times 100. This is the figure shown as “Occupancy Rate” / “Bed Utilization” on the Executive Overview and Patient Insights pages (91.95% in the delivered dashboards).
- **Readmission Rate** — `value_counts()` was used on `patient_id` to find how many patients appear more than once in the dataset (readmitted patients). Readmission Rate is then `readmitted_patients ÷ total unique patients`, multiplied by 100.

Why These KPIs Were Chosen

Each KPI was selected because it answers a specific operational or clinical question that a hospital administrator would realistically ask. Average Length of Stay and Readmission Rate speak to clinical quality and discharge efficiency; Bed Utilization and Occupancy Rate speak to capacity planning and whether a department is over- or under-stretched; Total Admissions gives the baseline volume against which every other KPI is contextualized. Together, they form the five headline scorecards that appear at the top of every dashboard page in Modules 5–7.

Deliverables

- `KPI_Engineering.ipynb` — KPI calculation notebook
- `Hospital_Operations_Final_Dataset` (KPI-enriched) — dataset optimized for Tableau/Power BI import

Evaluation / Outcome

- All KPIs calculated and cross-verified against the source data (6+ KPIs generated, exceeding the project target).

- Final dataset structured and optimized for direct use in Tableau and Power BI.

Tools & Libraries Used

KPI engineering was carried out in a dedicated notebook, `KPI_Engineering.ipynb`, kept separate from the Module 2 cleaning notebook so that KPI logic could be revisited or extended without re-running the full cleaning pipeline each time.

- **pandas .describe()** — used on `length_of_stay` to review mean, min, max and quartiles before it was trusted as a headline KPI.
- **Series.value_counts()** — used on `patient_id` to identify readmitted patients for the Readmission Rate calculation.
- **Simple arithmetic on aggregated columns** — used for every rate-based KPI, kept deliberately simple so the formulas stay auditable and easy to explain.

Validation Approach

Every KPI was cross-checked at least twice before being written into the final dataset: once against a manual calculation on a small sample of rows, and once by comparing the KPI's value when computed hospital-wide against the same KPI recomputed from the service-level table. For example, the overall Bed Utilization Rate and the Occupancy Rate calculated from the de-duplicated service-level KPI table were confirmed to agree at 91.95%, which gave confidence that the service-level aggregation in Module 2 had not introduced any double-counting.

- **Sample-level manual checks** — a handful of patient records were traced by hand from raw admission/discharge dates through to the final `length_of_stay` value.
- **Cross-table agreement checks** — hospital-wide KPIs were compared against the same KPI rebuilt from the service-level table.
- **Range sanity checks** — every rate-based KPI (Bed Utilization, Occupancy, Readmission) was confirmed to fall between 0% and 100% before being finalized.

Challenges & Resolutions

The trickiest part of this module was making sure a KPI calculated two different ways — for example Bed Utilization computed directly versus Occupancy Rate computed from the already-aggregated service-level table — produced the same answer. Small differences here would have been easy to miss but would have undermined trust in the dashboards.

- **Divergent results between two calculation paths for the same KPI** — resolved by tracing both formulas back to the same underlying `available_beds` and `patients_admitted` columns.
- **Readmission Rate initially inflated by visit-level duplicates** — resolved by counting unique `patient_id` occurrences rather than raw rows.
- **Keeping KPI formulas simple enough to explain to a non-technical audience** — prioritized over more statistically elaborate alternatives, in line with the project's dashboard-first goal.

Module 4: Dashboard Planning & Prototyping

Milestone 2 — KPI Engineering and Dashboard Planning

Before any chart was built in Tableau or Power BI, the full dashboard suite was wireframed and planned in a dedicated blueprint deck, *Hospital_Operations_Patient_Analytics_Blueprint.pptx*. This step defined, page by page, exactly which KPIs, filters, and visual types would appear — so that development in Modules 5–7 was a matter of building to an agreed plan rather than designing on the fly.

Wireframe Structure

The blueprint defines four dashboard pages, each following the same layout pattern: a left-hand navigation rail listing all four pages, a filter panel at the top, five KPI scorecards, and a set of supporting charts underneath.

Page 1 — Hospital Patient Analytics

- Filters planned: Admission Date, Department, Condition, Gender, Insurance
- KPIs planned: Total Patients, Admissions, Avg LOS, Bed Utilization, Satisfaction
- Visuals planned: Patient Admissions Trend (line chart), Patient Distribution by Department (bar chart), Condition/Diagnosis Mix (donut chart), Operational Performance Scorecard (matrix)

Page 2 — Service Performance

- Filters planned: Month/Year, Service, Department, Patient Type, Outcome
- KPIs planned: Total Requests, Admitted, Refused, Avg Wait Time, Service Satisfaction
- Visuals planned: Service Demand Trend (line chart), Admission Conversion by Service — Requests/Admitted/Refused (bar chart), Service Performance/Outcome (funnel chart), Service Scorecard (matrix), Age vs Satisfaction/Cost (scatter chart)

Page 3 — Staff Management

- Filters planned: Month/Year, Department, Staff Role, Shift, Service
- KPIs planned: Total Staff, Avg Attendance, Staff Utilization, Avg Workload, Staff Satisfaction
- Visuals planned: Staff Attendance Trend (line chart), Staff Distribution by Department (bar chart), Workload vs Satisfaction (scatter chart), Staff Performance Scorecard (matrix)

Page 4 — Financial Overview

- Filters planned: Year, Department, Service, Insurance, Payment Status
- KPIs planned: Total Revenue, Total Cost, Avg Patient Cost, Insurance Claims, Profit/Margin
- Visuals planned: Revenue Trend (line chart), Revenue by Department/Service (bar chart), Cost Distribution (donut chart), Financial Performance Scorecard (matrix), Insurance/Claim Status — claimed vs not claimed (donut chart)

Planning Process

- Sketched each page as a low-fidelity structural wireframe (labelled boxes for filters, KPIs and chart placeholders) before connecting any real data, so the layout could be reviewed on its own merits.
- Chose one dominant chart type per question — trend questions were mapped to line charts, comparison questions to bar charts, part-to-whole questions to donut charts, and relationship questions to scatter charts — to keep each visual immediately readable.

- Defined a consistent navigation rail (the four page names) and a consistent filter-panel position across all four pages, so that switching pages would feel seamless during the build phase.
- Reviewed the wireframes against the KPIs engineered in Module 3, confirming that every planned KPI card and chart had a corresponding column already available in the final dataset.

Deliverables

- Hospital_Operations_Patient_Analytics_Blueprint.pptx — four-page dashboard storyboard / wireframe deck

Evaluation / Outcome

- Dashboard layout, KPI placement and chart types approved before development began.
- Wireframes used as the direct blueprint for the Tableau and Power BI builds in Modules 5–7.

Tools Used for Wireframing

The wireframes were built directly in PowerPoint rather than a dedicated wireframing tool, which made it straightforward to reuse the same shapes, colour palette and grid layout that would later appear in the Tableau and Power BI builds. Keeping the wireframe deck in .pptx format also made it easy to share with a reviewer for sign-off before any development time was spent in Tableau or Power BI.

- **PowerPoint shapes and placeholders** — used to lay out the navigation rail, filter panel and chart placeholders for each of the four pages.
- **A consistent colour palette (teal headers, neutral greys for placeholder boxes)** — chosen early so the wireframes would visually resemble the eventual dashboards.
- **Speaker notes on each slide** — used to record which KPI or chart type each placeholder represented, doubling as a build checklist for Modules 5 through 7.

Design Principles Applied

Three simple principles guided every wireframing decision: consistency, meaning the same navigation rail and filter-panel position on all four pages; proportional emphasis, meaning KPI scorecards placed above supporting charts since headline numbers matter most to a first-time viewer; and one-chart-one-question, meaning each visual was designed to answer a single, specific question rather than being a general-purpose chart.

- **Consistency** — identical page structure across all four dashboard pages so users could navigate intuitively.
- **Proportional emphasis** — KPI scorecards given the most prominent position at the top of each page.
- **One-chart-one-question** — each visual mapped to exactly one planned question — trend, comparison, part-to-whole or relationship.
- **Reviewability** — every wireframe element was cross-checked against the KPIs engineered in Module 3 before being approved.

Module 5: Build Patient Analytics Dashboard (Tableau & Power BI)

Milestone 3 — Dashboard Development

This module built the first dashboard page of the suite, focused on patient-level analytics and an executive-level overview of hospital performance, developed in parallel on both Tableau (“Patient Analytics” tab) and Power BI (“Executive Overview” tab).

Tasks Performed

- Built KPI cards for Total Patients, Average Length of Stay, Patient Satisfaction, Total Cost, and Total Admissions.
- Created “Admission by Condition” and “Top Conditions by Total Cost” visuals.
- Built a geographic “Patients by State” map visual.
- Created gender-wise insurance-claim and monthly admission trend charts.
- Added interactive filters for Month, Gender, Condition, and State.
- Replicated the same analytical view as an Executive Overview page in Power BI, with Patient Journey, Service Analytics, and Patient Insights navigation tabs.

Deliverables

- Hospital_operation_&_patient_analytics.twb (Module 5 Tableau)
- HOPA.pbix (Module 5 Power BI)

Outputs

Tableau Dashboard:

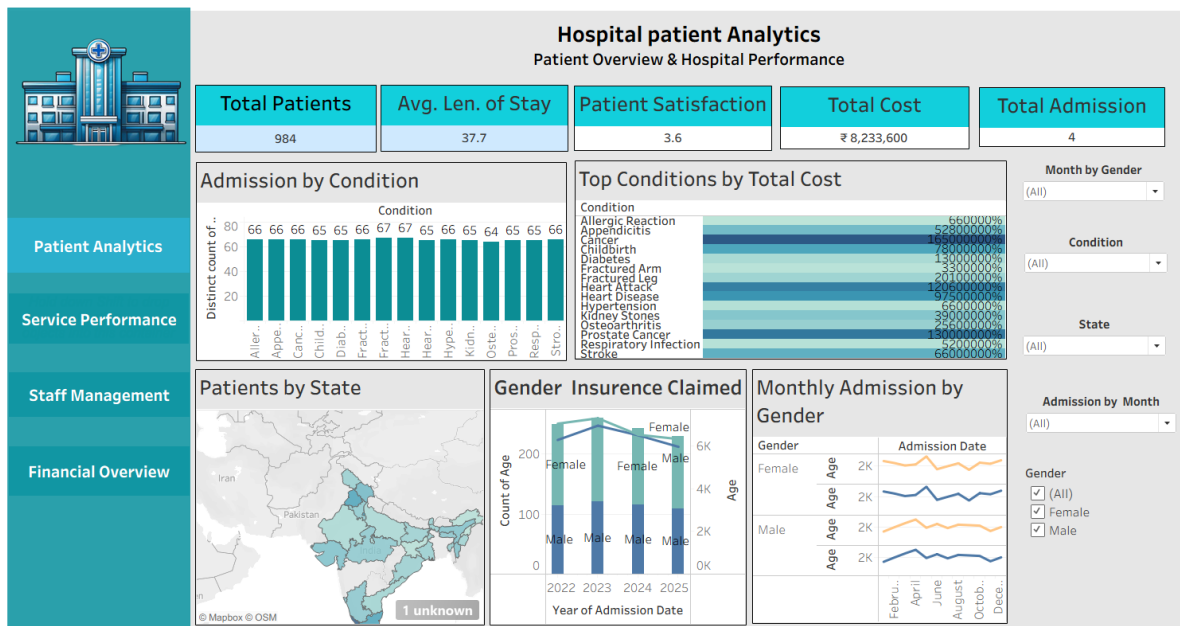


Fig 5.1 — Tableau: Patient Analytics dashboard

Power BI Dashboard:

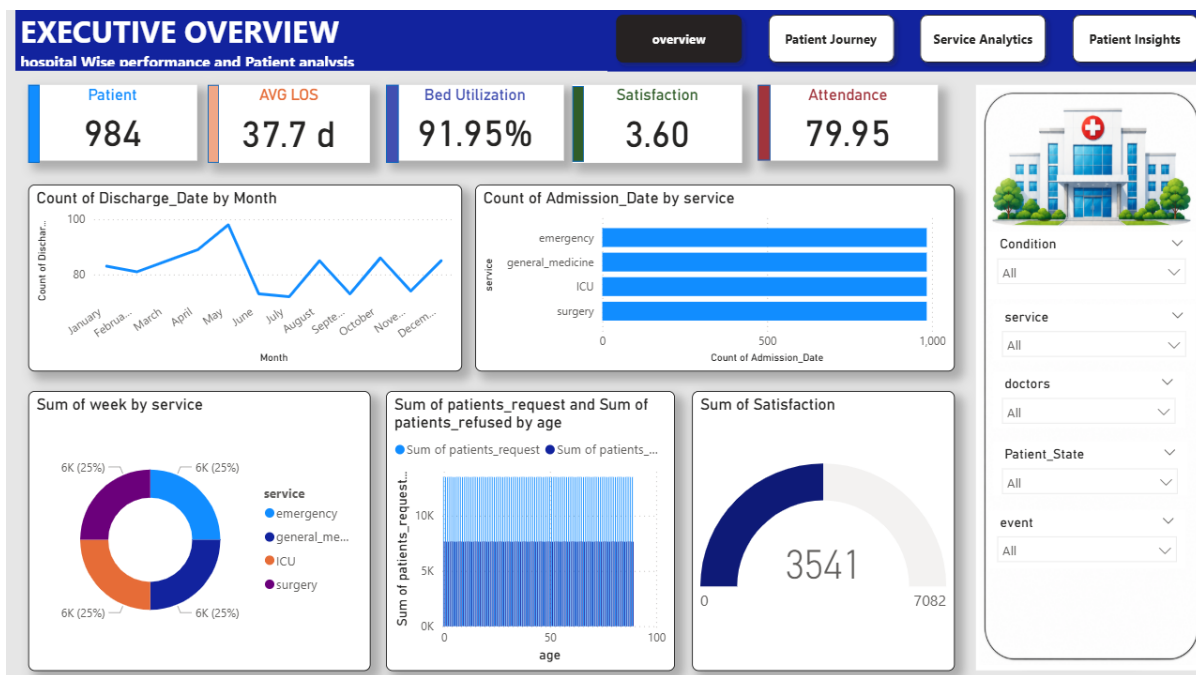


Fig 5.2 — Power BI: Executive Overview dashboard

Chart-by-Chart Explanation — Tableau (Patient Analytics)

KPI Scorecards (5 single-value cards) — Total Patients (984), Avg. Length of Stay (37.7 days), Patient Satisfaction (3.6), Total Cost (₹8,233,600) and Total Admission (4) are shown as large single numbers at the top of the page, giving viewers an instant summary of overall hospital performance before they drill into any detail.

Admission by Condition (Vertical bar chart) — Plots the distinct count of patients for each medical condition (Allergic Reaction, Appendicitis, Cancer, Childbirth, Diabetes, etc.). Because each condition shows a similarly-sized bar, it is used to quickly confirm which conditions make up the bulk of admissions and whether any single condition dominates patient volume.

Top Conditions by Total Cost (Horizontal bar / highlight table) — Ranks every condition by total treatment cost, with a colour gradient (darker shading = higher cost) layered over the bars. This view is used to identify the costliest conditions — Cancer and Heart Attack stand out with the largest total spend — so that cost-control efforts can be targeted.

Patients by State (Filled (choropleth) map) — Shades each Indian state by the number of patients treated, using Tableau's built-in mapping. It is used to see the geographic spread of the patient base and to support regional resource-planning decisions.

Gender Insurance Claimed (Dual-axis combo chart (bar + line)) — Compares the number of insurance claims made by female and male patients across admission years 2022–2025. The combination of bars and an overlaid line makes it easy to see both the absolute claim volume and the year-over-year trend for each gender at the same time.

Monthly Admission by Gender (Small-multiple line charts) — Splits admission counts by month into two stacked line panels — one for female patients and one for male patients — so seasonal admission patterns can be compared side by side for each gender.

Filter Panel (Drop-downs and checklists) — Month by Gender, Condition and State drop-downs, plus an Admission-by-Month drop-down and a Gender checklist, let a viewer slice every chart on the page interactively without needing a new dashboard.

Chart-by-Chart Explanation — Power BI (Executive Overview)

KPI Cards (*5 single-value cards*) — Patient (984), AVG LOS (37.7d), Bed Utilization (91.95%), Satisfaction (3.60) and Attendance (79.95) are repeated as headline cards on every Power BI report page, keeping the same reference numbers visible regardless of which page a viewer is on.

Count of Discharge_Date by Month (*Line chart*) — Tracks how many patients were discharged in each calendar month, used to monitor patient turnover and support discharge-planning decisions across the year.

Count of Admission_Date by service (*Horizontal bar chart*) — Compares total admission volume across Emergency, General Medicine, ICU and Surgery, immediately highlighting which department handles the highest patient load.

Sum of week by service (*Donut chart*) — Shows an even 25% split of scheduled service-weeks across the four departments — a quick sanity check that the underlying weekly data is balanced across services.

Sum of patients_request and Sum of patients_refused by age (*Combo bar/line chart*) — Overlays patient requests against patient refusals across different ages, used to flag age groups where capacity shortfalls (high refusals relative to requests) are occurring.

Sum of Satisfaction (*Radial gauge*) — Displays a cumulative satisfaction score (3,541 out of a possible 7,082) as a semi-circular gauge, giving a single “health-check” style read of overall patient experience.

Filter Panel (*Slicers*) — Condition, service, doctors, Patient_State and event slicers sit in the right-hand rail, allowing every visual on the page to be filtered interactively.

Evaluation / Outcome

- Interactive visualizations operational on both platforms.
- Patient-level KPIs validated against the engineered KPI dataset from Module 3.

Tools & Techniques Used

This dashboard page was built twice, once in Tableau and once in Power BI, using each tool's native strengths rather than forcing an identical build in both — Tableau's mapping and highlight-table features for the geographic and cost visuals, and Power BI's cards, slicers and gauge for the same headline story.

- **Tableau calculated fields** — used to derive the Total Cost and Length of Stay measures shown on the KPI scorecards.
- **Tableau's built-in geocoding** — used for the Patients by State filled map, requiring no external shapefiles.
- **Power BI DAX measures** — used to build the Bed Utilization and Attendance KPI cards shown across every report page.
- **Power BI slicers** — used to give the Executive Overview page the same interactivity as the Tableau filter panel.

Challenges & Resolutions

Keeping the Tableau and Power BI versions of this page visually and numerically consistent took more care than building either dashboard in isolation would have.

- **Slightly different default aggregations between Tableau and Power BI** — resolved by explicitly setting the aggregation — sum, average or distinct count — on every measure in both tools.
- **State-name mismatches for the map visual** — resolved by standardizing Patient_State values during the Module 2 cleaning step, before they reached either tool.
- **Keeping five KPI cards visually aligned** — resolved by fixing a shared card size and font scale as a small style guide.

Module 6: Build Staff Management & Service Analytics Dashboard (Tableau & Power BI)

Milestone 3 — Dashboard Development

This module built the second dashboard page, focused on staff performance, scheduling, and service-level operational analysis, developed on Tableau (“Staff Management” tab) and Power BI (“Service Analysis” tab).

Tasks Performed

- Built KPI cards for Total Staff, Patient Rate, Satisfaction Rate, Staff Rate, and Staff Morale.
- Created an outlier/box-plot visual to flag unusual event volumes.
- Built “Staff by Role” and “Year by Service” visuals to analyze staffing distribution over time.
- Created monthly service and yearly admission trend charts.
- Built the Power BI equivalent “Service Analysis” page with patient-request trends, satisfaction-by-service comparisons, and a detailed staff roster table.
- Added filters for Admission Year, Service, and Length of Stay.

Deliverables

- Hospital_operation_&_patient_analytics.twb (Module 6 Tableau)
- HOPA.pbix (Module 6 Power BI)

Outputs

Tableau Dashboard:

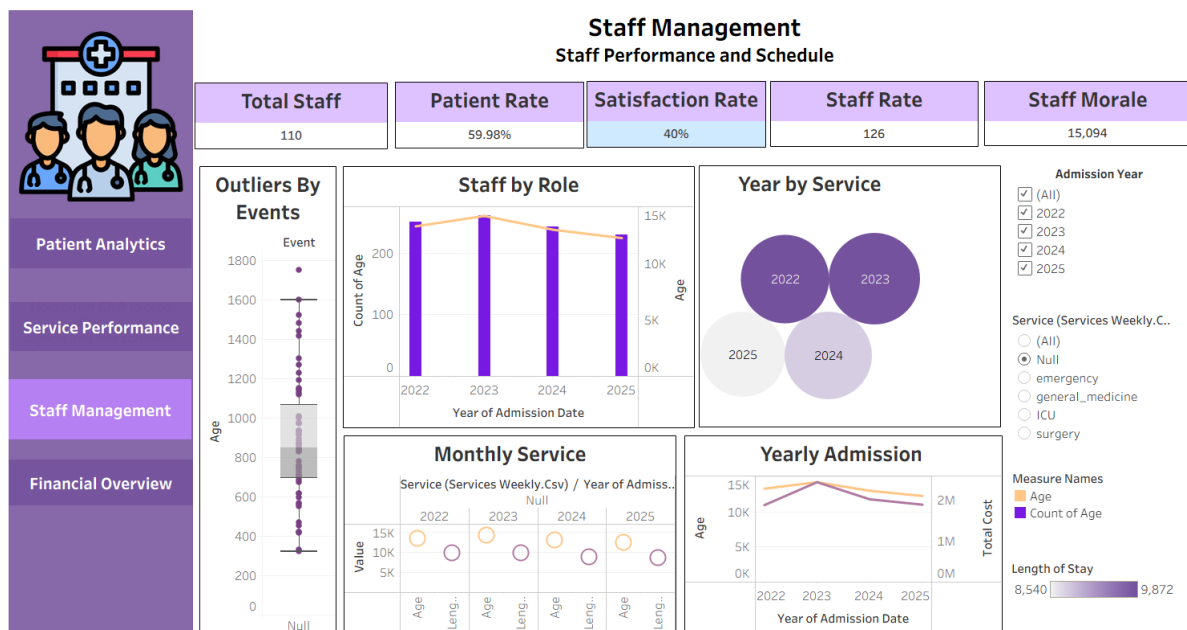


Fig 6.1 — Tableau: Staff Management dashboard

Power BI Dashboard:

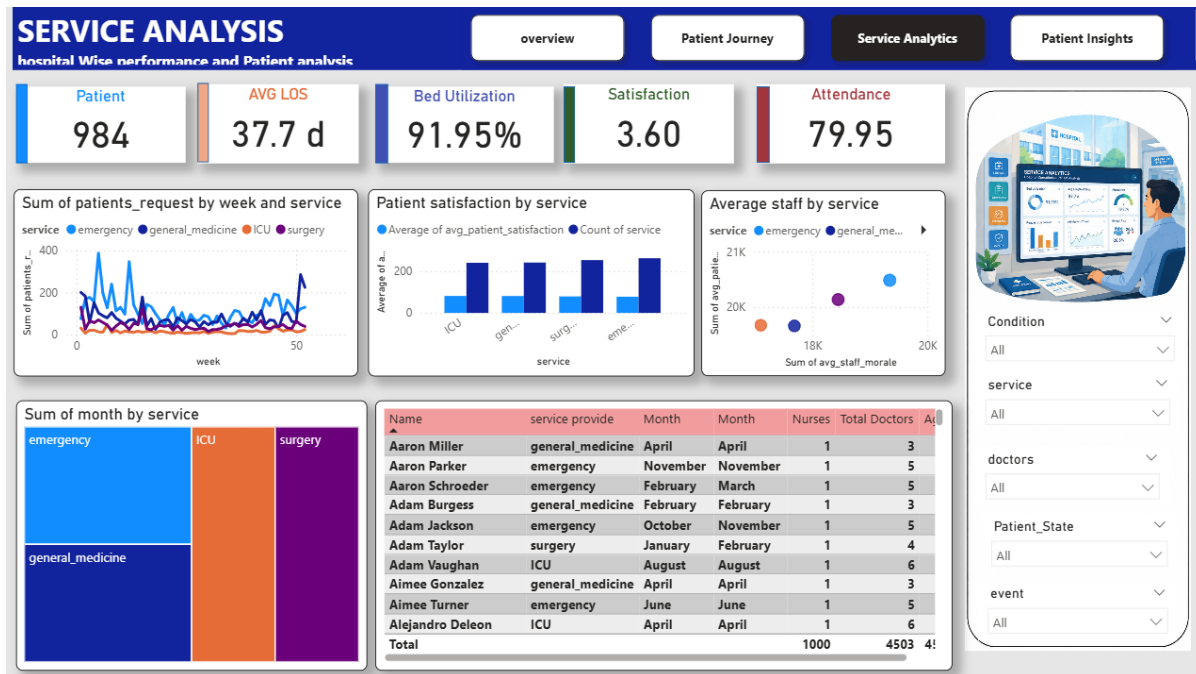


Fig 6.2 — Power BI: Service Analysis dashboard

Chart-by-Chart Explanation — Tableau (Staff Management)

KPI Scorecards (5 single-value cards) — Total Staff (110), Patient Rate (59.98%), Satisfaction Rate (40%), Staff Rate (126) and Staff Morale (15,094) summarize workforce scale and wellbeing at a glance.

Outliers by Events (Box-and-whisker plot) — Plots the distribution of Age against recorded events, with individual data points overlaid on the box plot. It is used to detect outlier ages associated with unusually high or low event counts that would not be obvious from an average alone.

Staff by Role (Combo bar + line chart) — Tracks staff counts by role across the years 2022–2025, with bars showing yearly volume and a line overlay tracing the trend, making it easy to see whether staffing levels are growing, shrinking, or holding steady.

Year by Service (Packed bubble chart) — One bubble per year (2022, 2023, 2024, 2025), sized by activity volume, giving an immediate visual comparison of which years were busiest for a given service without needing to read exact axis values.

Monthly Service (Dot-matrix / circle-view table) — Cross-tabulates month against year for each service, with circle size and colour encoding the value in each cell — used to compare service load month-by-month rather than only at a yearly level.

Yearly Admission (Dual-axis line chart) — Plots Age and Total Cost trends together across 2022–2025 on separate axes, allowing a viewer to see at a glance whether cost and patient age are moving in the same direction over time.

Filter Panel (Checklists, radio buttons and a range slider) — Admission Year checklist, Service radio buttons, Measure Names checkboxes and a Length-of-Stay range slider give fine-grained control over which staff and service records are displayed.

Chart-by-Chart Explanation — Power BI (Service Analysis)

KPI Cards (5 single-value cards) — The same global headline metrics (Patient, AVG LOS, Bed Utilization, Satisfaction, Attendance) are repeated here for consistency with the other report pages.

Sum of patients_request by week and service (*Multi-line chart*) — Plots one line per department (Emergency, General Medicine, ICU, Surgery) across 52 weeks, used to track weekly demand fluctuation and spot spikes in patient requests for a specific service.

Patient satisfaction by service (*Combo bar + line chart*) — Compares average patient satisfaction against service volume for ICU, General Medicine, Surgery and Emergency side by side, highlighting whether higher-volume departments are also the ones with lower satisfaction.

Average staff by service (*Scatter plot*) — Plots average staff morale against average patient satisfaction, with one bubble per department, used to explore whether departments with happier staff also report higher patient satisfaction.

Sum of month by service (*Treemap*) — Represents monthly service volume for Emergency, ICU, Surgery and General Medicine as proportionally-sized rectangles, giving an immediate visual read of which department consumes the largest share of monthly activity.

Staff Roster Table (*Detailed matrix / table*) — Lists every staff member's name, service, scheduled month, worked month, nurse count and doctor count, giving operational teams a drill-down view behind the summary charts.

Filter Panel (*Slicers*) — Condition, service, doctors, Patient_State and event slicers allow the whole page to be filtered interactively, consistent with the other Power BI pages.

Evaluation / Outcome

- Staffing and service-level visuals operational and cross-checked with the staff and schedule datasets.
- Department/service comparisons validated across both tools.

Tools & Techniques Used

The staffing and service-analytics page leaned more heavily on statistical chart types than the other pages, since its job is to surface anomalies, such as unusual event volumes or morale outliers, rather than simply report totals.

- **Tableau box-and-whisker plots** — used to detect outlier ages associated with unusual event counts.
- **Tableau packed bubble charts** — used to compare year-by-service activity volume at a glance.
- **Power BI scatter visuals** — used to explore the relationship between staff morale and patient satisfaction by department.
- **Power BI matrix/table visuals** — used for the detailed staff roster, giving a drill-down view behind the summary charts.

Challenges & Resolutions

Attendance data — 6,552 rows in staff_schedule.csv — was by far the largest and most repetitive table in the project, which made it easy for a naive aggregation to double-count staff who appeared under more than one role or service over the course of the year.

- **Risk of double-counting staff across weeks** — resolved by aggregating attendance to one row per staff member per service before computing attendance_rate.
- **Outlier ages skewing the average-based charts** — resolved by using a box-plot view instead of a simple bar average, so outliers remain visible rather than hidden.
- **Keeping the Tableau and Power BI staffing KPIs numerically aligned** — resolved the same way as Module 5, via a shared aggregation convention.

Module 7: Build Financial Overview & Patient Insights Dashboard (Tableau & Power BI)

Milestone 3 — Dashboard Development

This module built the third and final dashboard page, covering financial performance and deeper patient-outcome insights, completing the four-part integrated dashboard suite. It was developed on Tableau (“Financial Overview” tab) and Power BI (“Patient Insights” tab).

Tasks Performed

- Built KPI cards for Healthcare Cost, Insurance Claimed, Average Operation Cost, Total Patients, and Discharge Rate.
- Created “Cost by Condition” trend and “Cost by State” map visuals.
- Built insurance-by-gender and insurance-by-year breakdown visuals/tables.
- Built the Power BI “Patient Insights” page with Length of Stay by State, Bed Utilization by service, and Outcome-by-Condition/Gender visuals, including a Sankey-style flow of patient outcomes.
- Applied custom icon artwork to enhance the visual identity of each dashboard page.
- Linked global filters (Condition, Service, Admission Year) consistently across all four dashboard pages in both tools.

Deliverables

- Hospital_operation_&_patient_analytics.twb (Module 7 Tableau)
- HOPA.pbix (Module 7 Power BI)

Outputs

Tableau Dashboard:

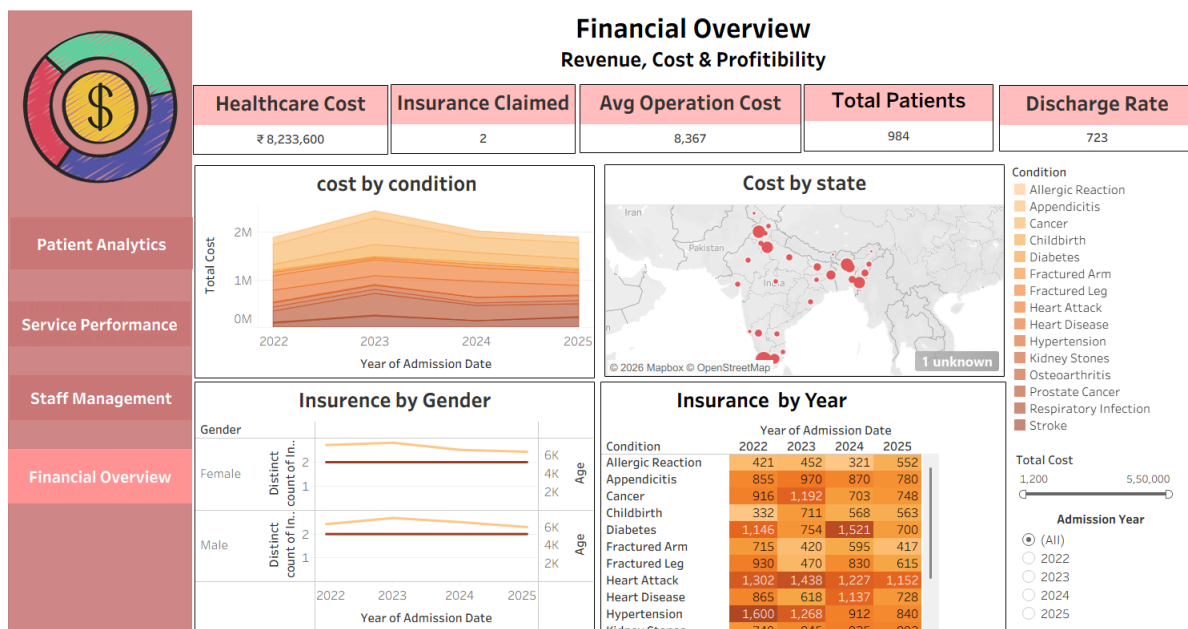


Fig 7.1 — Tableau: Financial Overview dashboard

Power BI Dashboard:

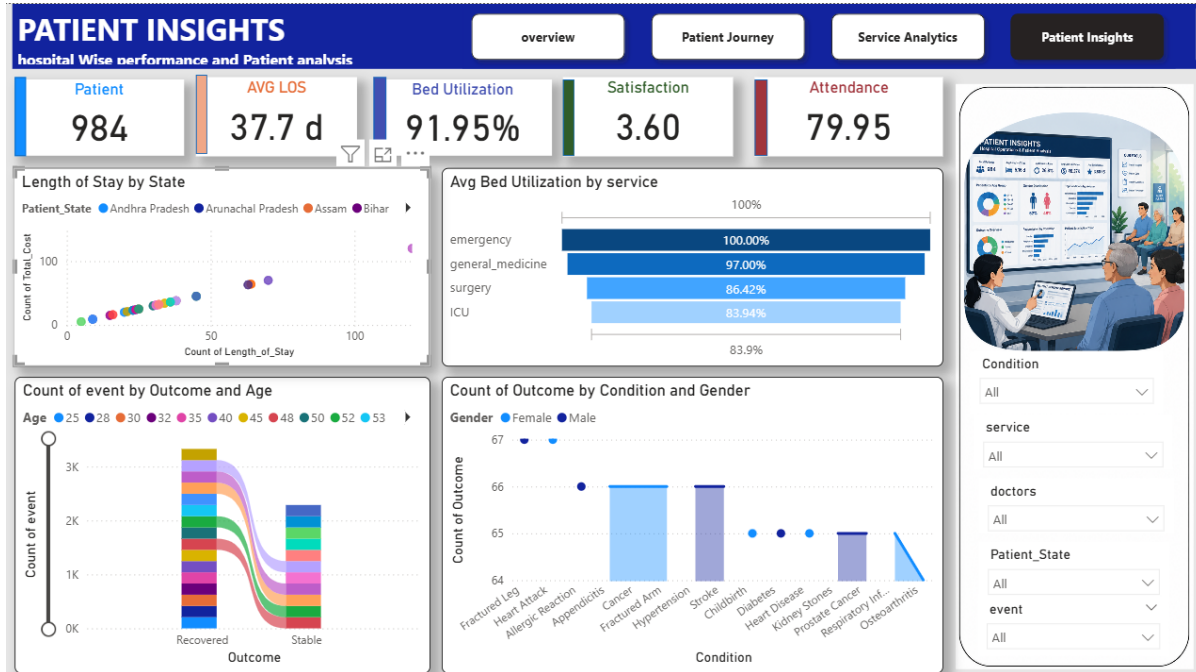


Fig 7.2 — Power BI: Patient Insights dashboard

Chart-by-Chart Explanation — Tableau (Financial Overview)

KPI Scorecards (5 single-value cards) — Healthcare Cost (₹8,233,600), Insurance Claimed (2), Avg Operation Cost (8,367), Total Patients (984) and Discharge Rate (723) summarize the financial position of the hospital at a glance.

Cost by Condition (Stacked area chart) — Shows cumulative treatment cost by condition across 2022–2025, letting a viewer track how the overall cost mix has evolved and which conditions are contributing the most to rising costs over time.

Cost by State (Proportional-symbol (bubble) map) — Places a circle on each Indian state, sized according to total treatment cost incurred there, immediately highlighting the states that drive the most healthcare spend.

Insurance by Gender (Dual-line chart) — Compares insurance-claim trends for female versus male patients year over year, used to check whether claim behaviour differs meaningfully by gender.

Insurance by Year (Cross-tab / highlight table) — Lists every condition's insurance-claim counts by year (2022–2025) in a matrix, with heat-map shading (darker = higher) so that condition-year combinations with the highest claim volume stand out immediately — for example Heart Attack and Diabetes show some of the darkest cells.

Filter Panel (Checklist and sliders) — Condition checklist, a Total Cost range slider and an Admission Year radio-button control let a viewer narrow the financial view down to a specific cost band, year, or set of conditions.

Chart-by-Chart Explanation — Power BI (Patient Insights)

KPI Cards (5 single-value cards) — The same global headline metrics are repeated here to keep every Power BI page anchored to the same reference numbers.

Length of Stay by State (Scatter plot) — Plots Count of Total Cost against Count of Length of Stay, with each point coloured by Patient_State, used to see whether longer hospital stays correlate with higher treatment cost in particular states.

Avg Bed Utilization by service (*Horizontal ranked bar chart*) — Ranks departments by bed-utilization percentage — Emergency at 100%, General Medicine at 97%, Surgery at 86.42% and ICU at 83.94% — immediately showing which department is closest to full capacity and therefore under the most operational pressure.

Count of event by Outcome and Age (*Sankey (flow) diagram*) — Maps patients by age band into two outcome buckets, Recovered and Stable, with the width of each flow representing the number of patients following that path. This view is used to visualize how patient outcomes are distributed across different ages.

Count of Outcome by Condition and Gender (*Scatter / dot plot*) — Plots outcome counts against condition on the x-axis, coloured by gender, used to compare recovery and outcome patterns across different medical conditions for male versus female patients.

Filter Panel (*Slicers*) — Condition, service, doctors, Patient_State and event slicers complete the consistent filtering experience carried across all Power BI pages.

Evaluation / Outcome

- All four dashboard pages (Patient Analytics, Staff/Service, Financial Overview, Patient Insights) integrated with consistent navigation and filters.
- Financial and outcome KPIs validated against the source dataset.

Tools & Techniques Used

As the final dashboard page, this module also served as an integration checkpoint — global filters for Condition, Service and Admission Year were re-tested across all four dashboard pages in both Tableau and Power BI to confirm the whole suite behaved as a single, cohesive product rather than four independent pages.

- **Tableau proportional-symbol maps** — used for the Cost by State visual, sized by total treatment cost per state.
- **Tableau highlight tables with heat-map shading** — used for the Insurance by Year cross-tab.
- **A Power BI Sankey-style flow visual** — used to trace patient outcomes by age band.
- **Global filter linking** — configured so the Condition, Service and Admission Year filters behave identically on every page of both dashboards.

Key Learnings

Building the same four-page dashboard suite in two different tools reinforced how much of a good analytics product is design discipline rather than tool-specific skill — the wireframes from Module 4, the KPI formulas from Module 3, and the cleaned dataset from Module 2 mattered far more to the end result than any single chart-type choice in Tableau or Power BI.

- A well-planned wireframe from Module 4 made both dashboard builds significantly faster and kept them visually consistent.
- KPIs engineered once in Module 3 and reused everywhere prevented numbers drifting apart across pages or tools.
- Cleaning and merging the data thoroughly up front in Module 2 meant almost no rework was needed once dashboard development began.